BINV2150-A — Javascript : avancé

Séance 20 - Chiffrement des mots de passe

Haute École Léonard de Vinci

Pourquoi sécuriser les mots de passe

- Actuellement, mots de passe stockés en clair dans la "DB"
- N'importe qui qui a accès au serveur peut les lire
- ➡️ Un dev peut se faire passer pour n'importe qui
- ➡️ Un attaquant qui accède au serveur aussi

- ⚠️ Pas seulement cette application !
- Beaucoup d'utilisateurs réutilisent les mots de passe
- Avec l'accès au compte mail de quelqu'un, on peut prendre le contrôle de toute sa vie numérique
- ➡️ Utilisez un gestionnaire de mots de passe (1Password, LastPass, NordPass, ...)

Hachage cryptographique

- Hachage non-cryptographique : fonction qui résume des données complexes en un nombre
- Hachage cryptographique, ajoute les propriétés suivantes :
  - **Déterministe** : même entrée ➡️ même sortie
  - **Irreversible** : impossible de retrouver l'entrée à partir de la sortie
  - **Résistant aux collisions** : difficile de trouver deux entrées différentes donnant la même sortie
  - **Effet d'avalanche** : petite modification de l'entrée ➡️ sortie complètement différente
  - **Sortie de taille fixe** : quelle que soit la taille de l'entrée
- MD5, SHA-1, SHA-2 (SHA-256, SHA-512), ...

Hachage salé

- Les hashs sont une information sensible !
- Attaques possibles avec la liste de tous les hashs (rainbow table attack)
- Utilisation d'un **salt** : donnée aléatoire ajoutée à l'input d'un hash cryptographique
- Sans **salt** : deux utilisateurs avec le même mot de passe auraient le même hash
- Possible d'appliquer plusieurs fois la fonction de hachage, à chaque fois avec un **salt** différent

Librairie `bcrypt`

- Installation

```
npm i bcrypt
npm i @types/bcrypt -D
```

- Configuration

```
// import de la librairie
import bcrypt from "bcrypt";

// Nombre de fois où la fonction de hachage sera appelée
// + haut, + sécurisé et + long
const saltRounds = 10;
```

Génération et vérification de hash

- Génération

```
const hashedPassword: string = await bcrypt.hash(password, saltRounds);

// alternative synchrone, trop longue en réaction à une requête
const adminPassword = bcrypt.hashSync("admin", saltRounds);
```

- Vérification

```
const passwordMatch: boolean = await bcrypt.compare(password, hashedPassword);

// alternative synchrone, trop longue en réaction à une requête
if (bcrypt.compareSync(password, userDB.password))
```

Création des utilisateurs par défaut

```
// services/auths.ts

const saltRounds = 10;

const defaultUsers: User[] = [
  {
    id: 1,
    username: "admin",
    password: bcrypt.hashSync("admin", saltRounds),
  },
];
```

Création d'un nouvel utilisateur

```ts
// services/auths.ts

async function register(credentials: Credentials): Promise<boolean> {
  const userFound = getUser(credentials.username);
  if (userFound) return false; // User already exists

  const users = parse<User>(jsonDbPath, defaultUsers);

  const newId = users.length > 0 ? users[users.length - 1].id + 1 : 1;
  const hashedPassword = await bcrypt.hash(credentials.password, saltRounds);

  const newUser: User = {
    id: newId,
    username: credentials.username,
    password: hashedPassword,
  };
  users.push(newUser);
  serialize(jsonDbPath, users);

  return true;
}
```

Login de l'utilisateur

```ts
// services/auths.ts

async function login(credentials: Credentials): Promise<string | undefined> {
  const user = getUser(credentials.username);
  if (!user) return undefined; // User not found

  const passwordMatch = await bcrypt.compare(credentials.password, user.password);
  if (!passwordMatch) return undefined; // Invalid password

  const token = jwt.sign({ username: user.username }, jwtSecret, { expiresIn: lifetimeJwt });
  return token;
}
```

Routes

```ts
// routes/auths.ts

router.post("/register", async (req, res) => {
  const credentials: unknown = req.body;
  if (!isCredentials(credentials)) return res.sendStatus(400); // Invalid input

  const created = await register(credentials);
  if (!created) return res.sendStatus(409); // User already exists

  return res.sendStatus(201);
});

router.post("/login", async (req, res) => {
  const credentials: unknown = req.body;
  if (!isCredentials(credentials)) return res.sendStatus(400); // Invalid input

  const token = await login(credentials);
  if (!token) return res.sendStatus(401); // Invalid username or password

  return res.json({ token });
});
```